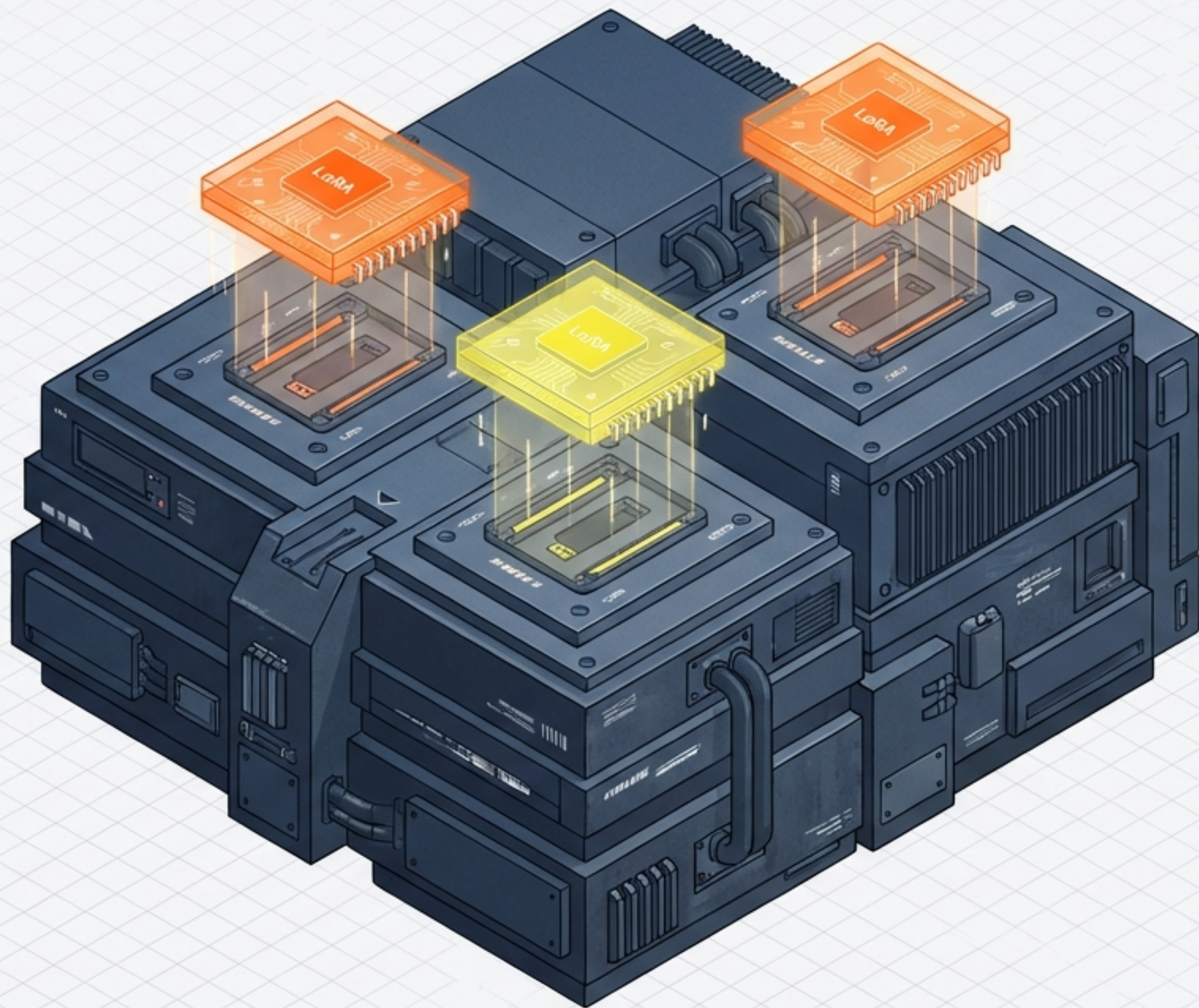


# 从零解构 LoRA: 大模型微调的 “降维打击”

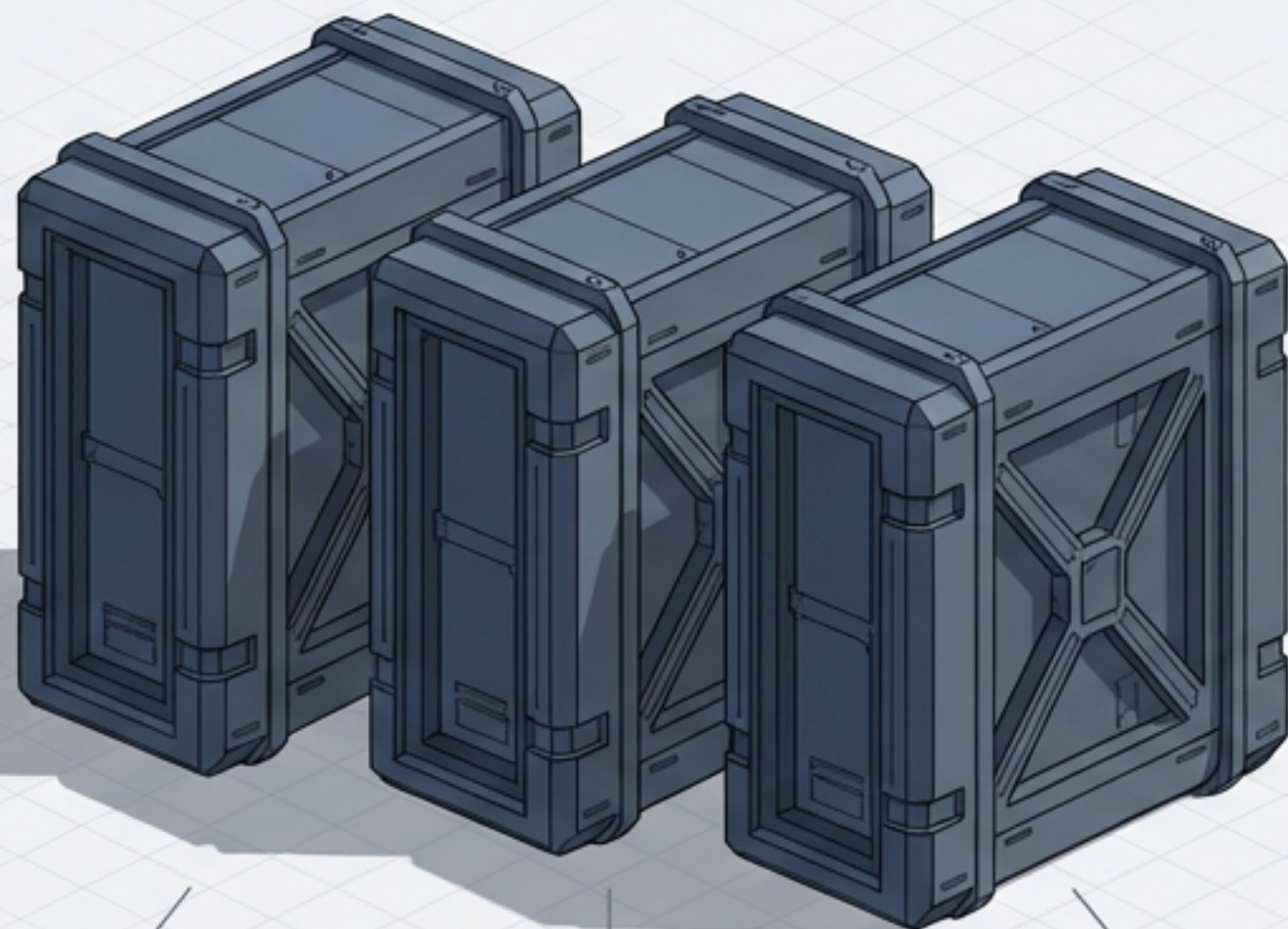
参数高效微调 (PEFT)  
核心逻辑与实战配置全指南

从 3 天到 3 小时，从全量重构到精准修补。





## 过去：全量微调



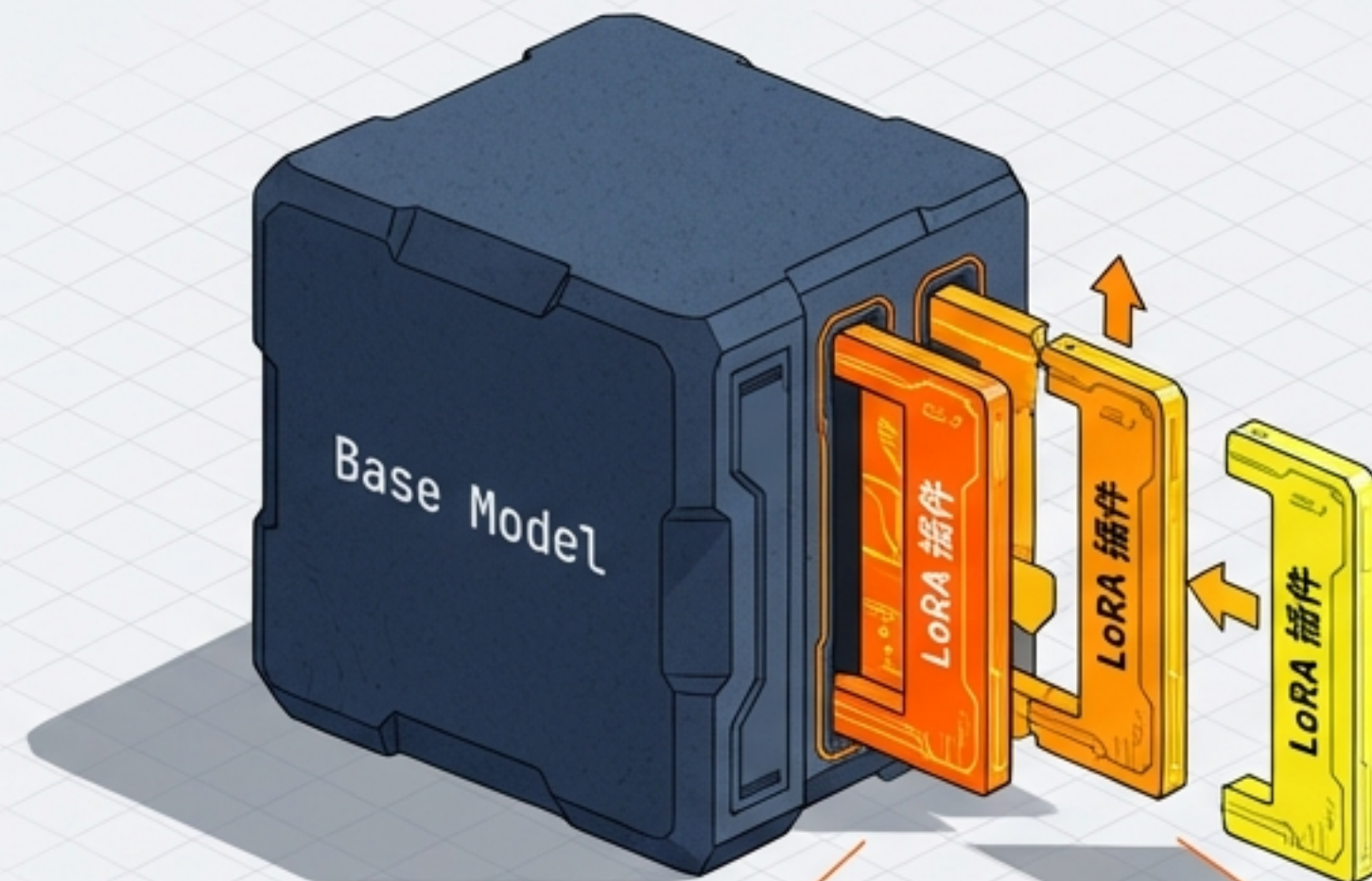
耗时 3 天

庞大显存占用

独立部署 3 个  
数百GB的大模型

针对“售前”、“售后”、“投诉”三个场景，必须从头训练并存储三个完整的模型副本。

## 现在：LoRA 插件化



耗时 3 小时

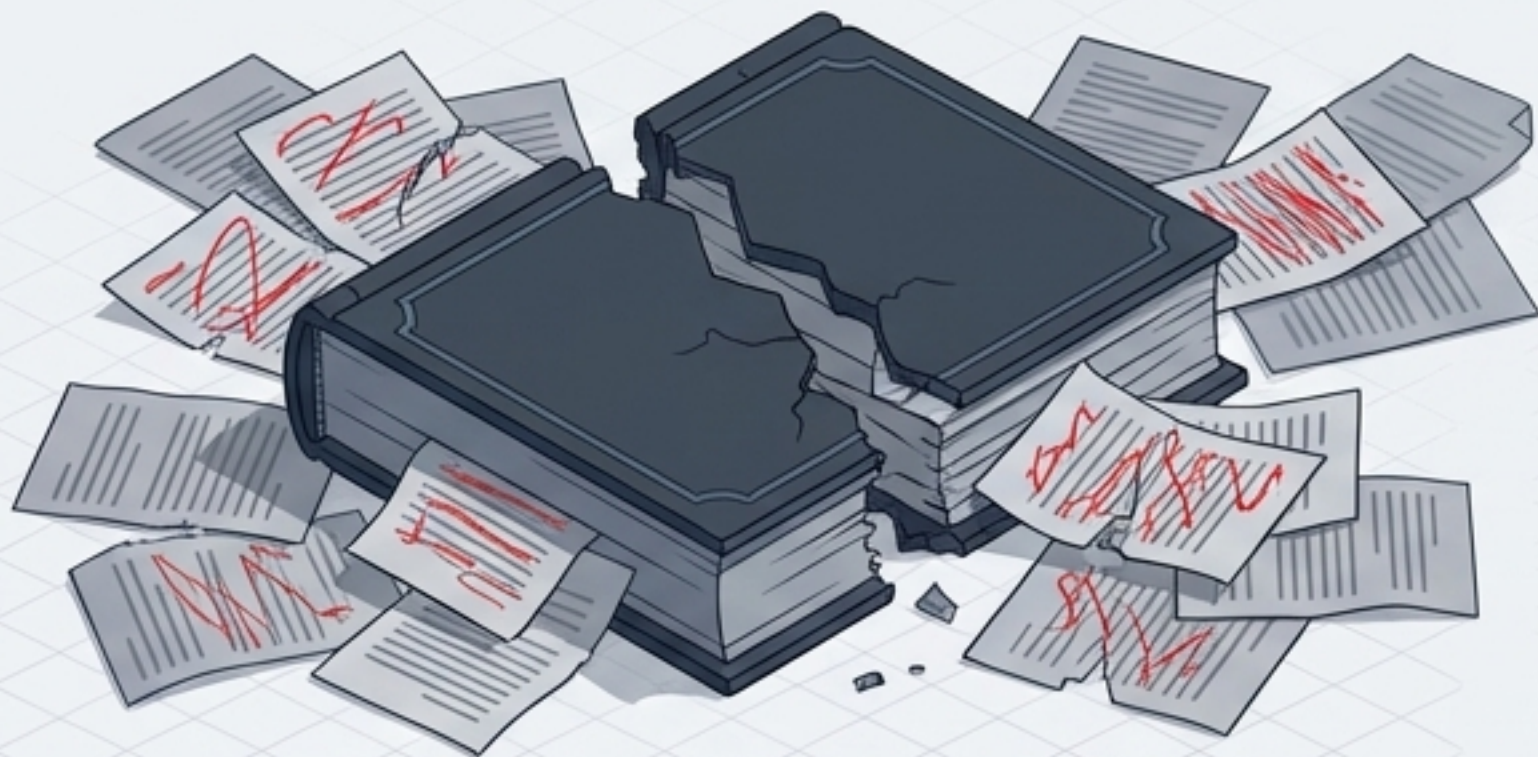
消费级 GPU 可运行

毫秒级热切换

1 个冻结的 Base 模型 + 3 个极小体积的 LoRA 插件，真正在工程落地中做到“花小钱办大事”。

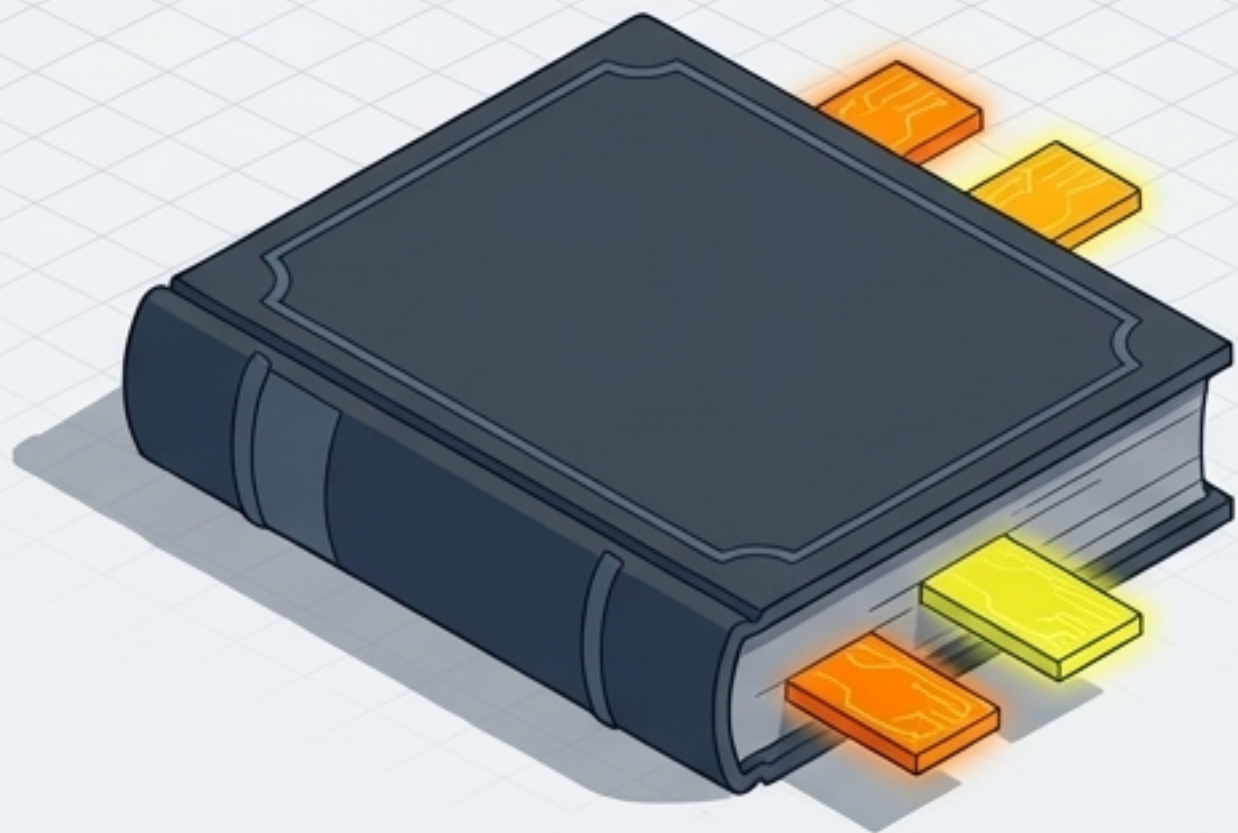


## 全量微调（重写全书）



改变原本结构，代价极其高昂。极易产生“灾难性遗忘”（例如为了学做菜，覆写了基础化学常识）。

## LoRA 微调（便利贴模式）



原书“一字不改”，仅在边缘附加低秩矩阵（例如调味口诀、食材搭配）。体积极小，即插即用，不干扰底层通用知识。



# 数学原理解构：为何能省99%参数？

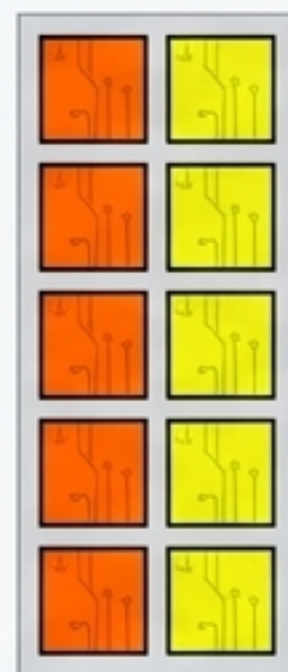
$$\Delta W = A \times B$$



需更新完整的增量矩阵

传统微调:  $\Delta W (5 \times 4) = 20$  个参数

=



降维矩阵 A ( $5 \times 2$ )

×



升维矩阵 B ( $2 \times 4$ )

=

LoRA:  $10 + 8 = 18$  个参数

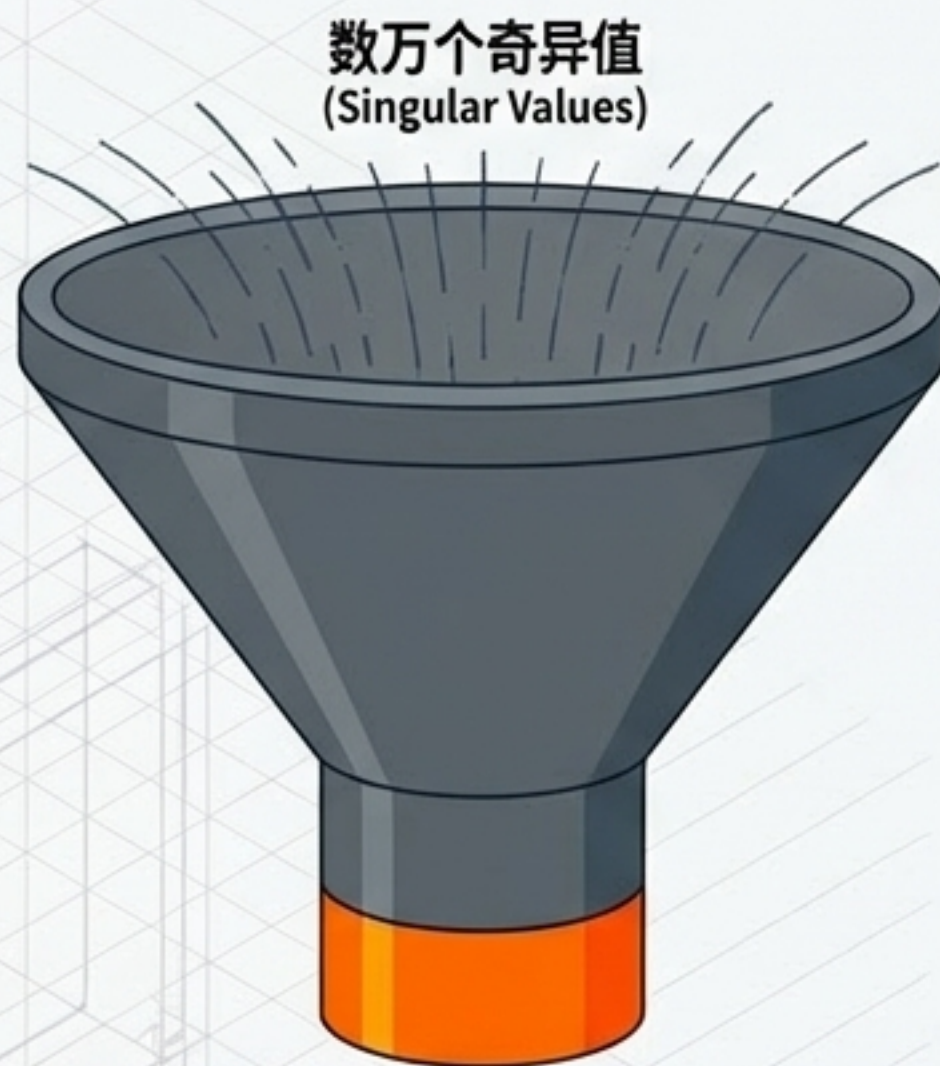
工业级震撼：在 7B 参数规模的大模型中，这种降维压缩让实际需要训练的参数量骤降至 0.01% - 3%。

如同改造大楼，无需强拆重建，只需在关键受力点加装钢筋支撑。

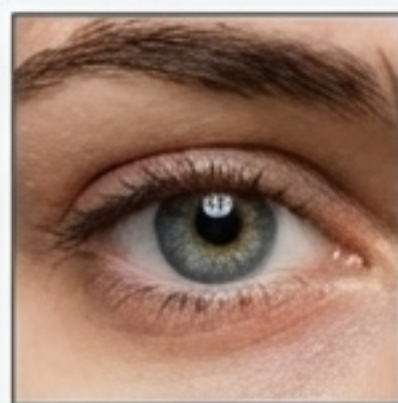


# 深入底层：增量矩阵为何是“低秩”的？

什么是秩（Rank）？矩阵中提供「独立新信息」的方向数量。

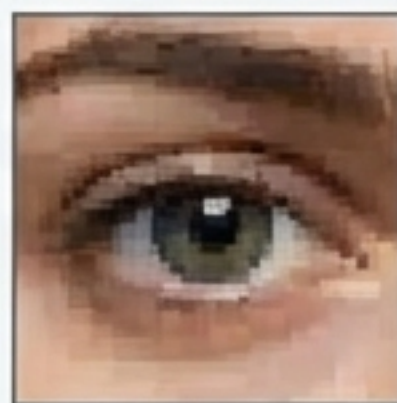


前 10-20 个奇异值占据  
90%+ 变化信息



高分辨率

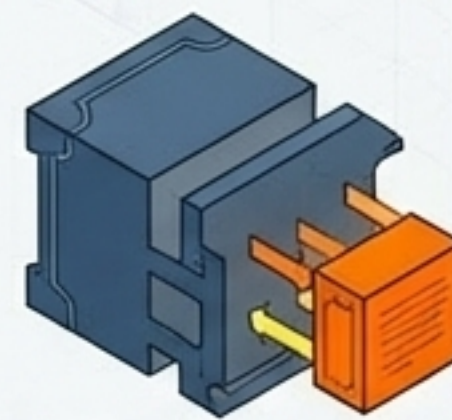
=



高压缩  
(无损失识别度)

## 结构化特化（调口音，非学语言）

增量变化并非随机，而是高度集中在任务相关方向。学习法律术语只触及合同/判决权重，绝不干扰“猫狗”等通用常识。



通用常识  
(General Knowledge)

法律术语  
(Legal Terms)

## 高效近似

仅需极少数核心维度，即可用 A 和 B 两个小矩阵完美重构任务主干变化，抛弃无用的噪音参数。



任务主干

=



A (核心维度)

×



噪音参数



# 灵魂拷问：为什么 Base Model 不能低秩分解？

## 预训练权重 $W$ （不可删减的灵魂）

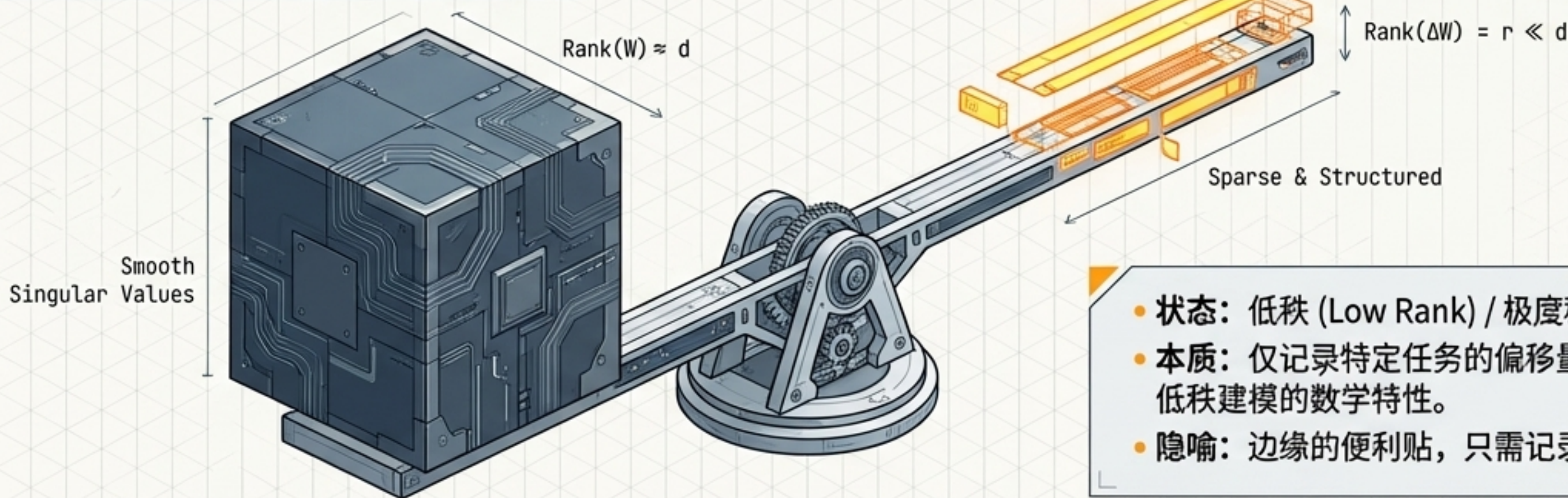
- 状态：满秩（Full Rank） / 奇异值分布平滑。
- 本质：蕴含海量且均匀的通用知识。强行低秩分解会导致严重的「知识坍塌」。
- 隐喻：百科全书的正文，一字不可省。

Source: Hen Sans SE

## 微调增量 $\Delta W$ （极致精简的插件）

- 状态：低秩（Low Rank） / 极度稀疏且结构化。
- 本质：仅记录特定任务的偏移量，完美契合低秩建模的数学特性。

JetBrains Mono



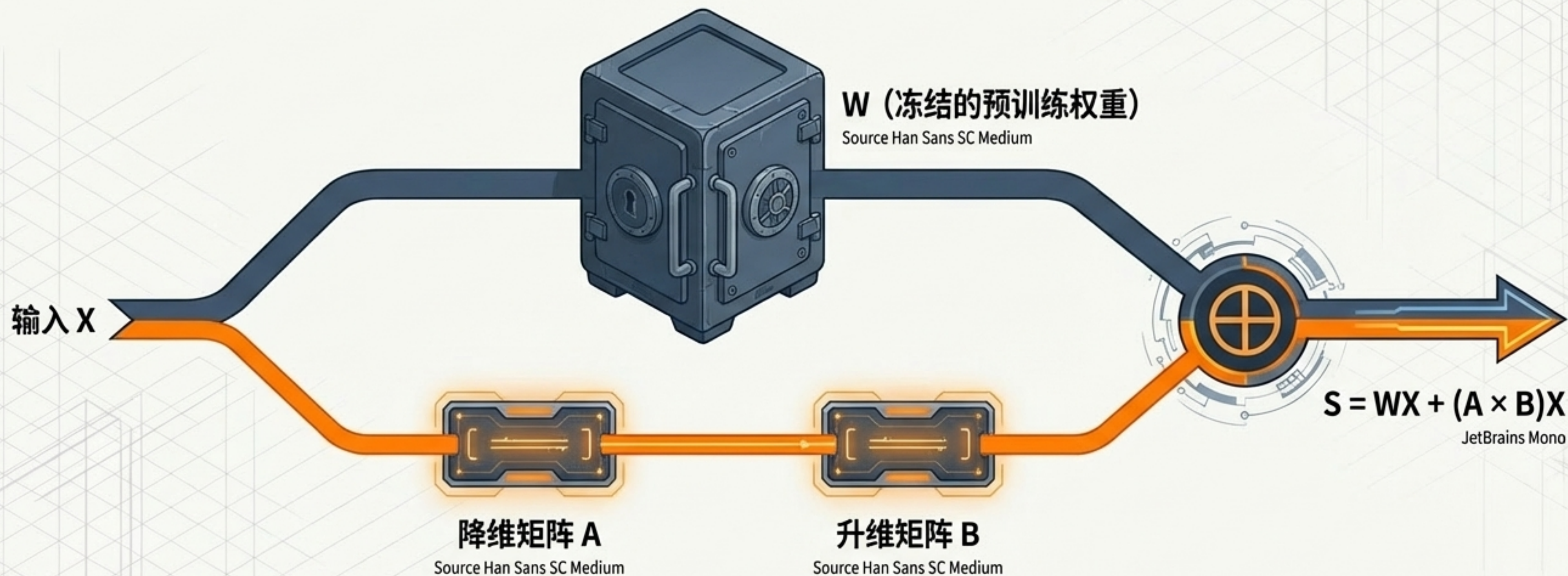
JetBrains Mono

- 状态：低秩 (Low Rank) / 极度稀疏且结构化。
- 本质：仅记录特定任务的偏移量，完美契合低秩建模的数学特性。
- 隐喻：边缘的便利贴，只需记录关键口诀。

**结论：**LoRA 不是在偷懒，而是完美契合了模型学习新任务时的底层数学特性。



# 架构透视：前向传播与无损融合



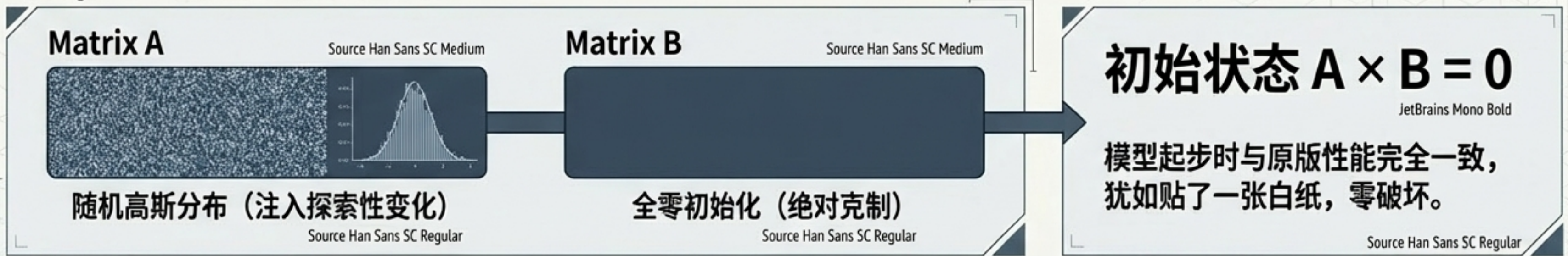
**核心优势：无损融合与 0 推理延迟**

在实际部署推理时，可以直接将  $A \times B$  预先计算并加到  $W$  中 ( $W_{\text{new}} = W + AB$ )，实现毫无额外性能损耗的极速推理。

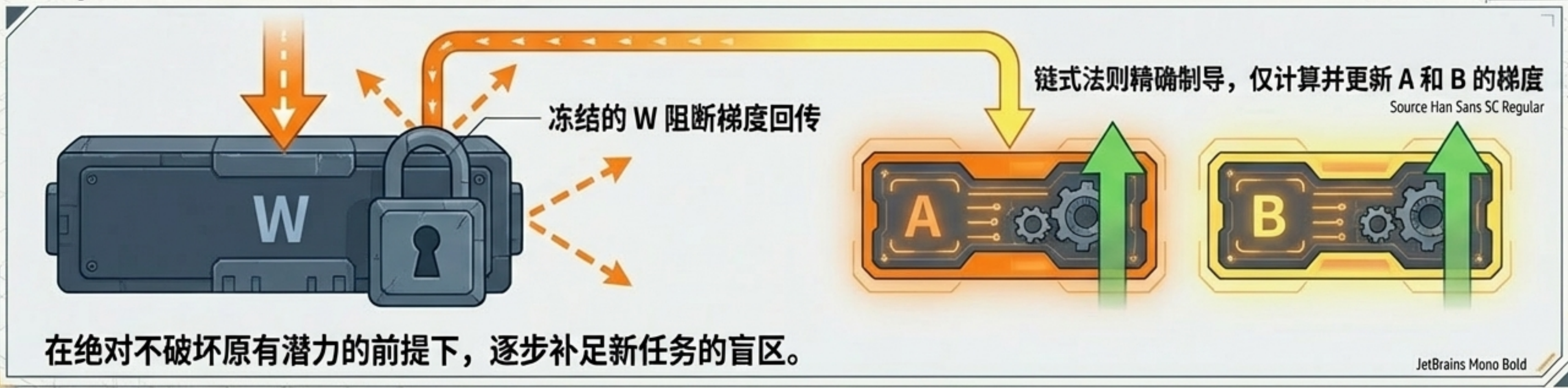


# 架构透视：反向传播与零干涉初始化

## Step 1: 巧妙的零干涉初始化



## Step 2: 反向传播（戴着镣铐跳舞）





# 战略价值：LoRA带来的四大工程红利

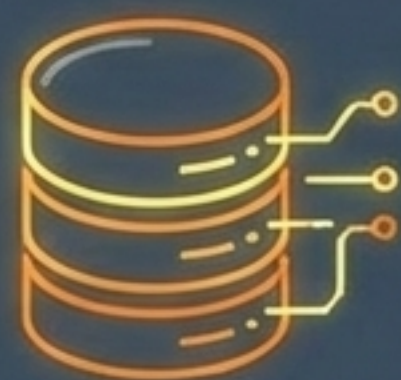


## 极简算力成本 (Cost)

在 GPT-3 级别实验中，仅训 0.01% 参数即可达全量微调 95%-99% 性能。普通消费级 GPU 即可入局，迭代极快。

Source Han Sans SC Medium

1px



## 模块化存储 (Storage)

告别动辄数百 GB 的模型副本。Base 模型仅需 1 份，LoRA 权重仅几 MB 至几十 MB，节省海量存储空间。

Source Han Sans SC Medium

1px



## 抗遗忘性 (Memory)

原模型权重绝对冻结，学英语绝不影响母语。避免灾难性遗忘，维持极高的通用泛化能力。

Source Han Sans SC Medium



## 毫秒级热切换 (Agility)

内存中动态加载/卸载不同 LoRA 权重。同一个基础大模型，瞬间在“代码助手”与“客服机器人”间无缝并发切换。

Source Han Sans SC Medium

1px

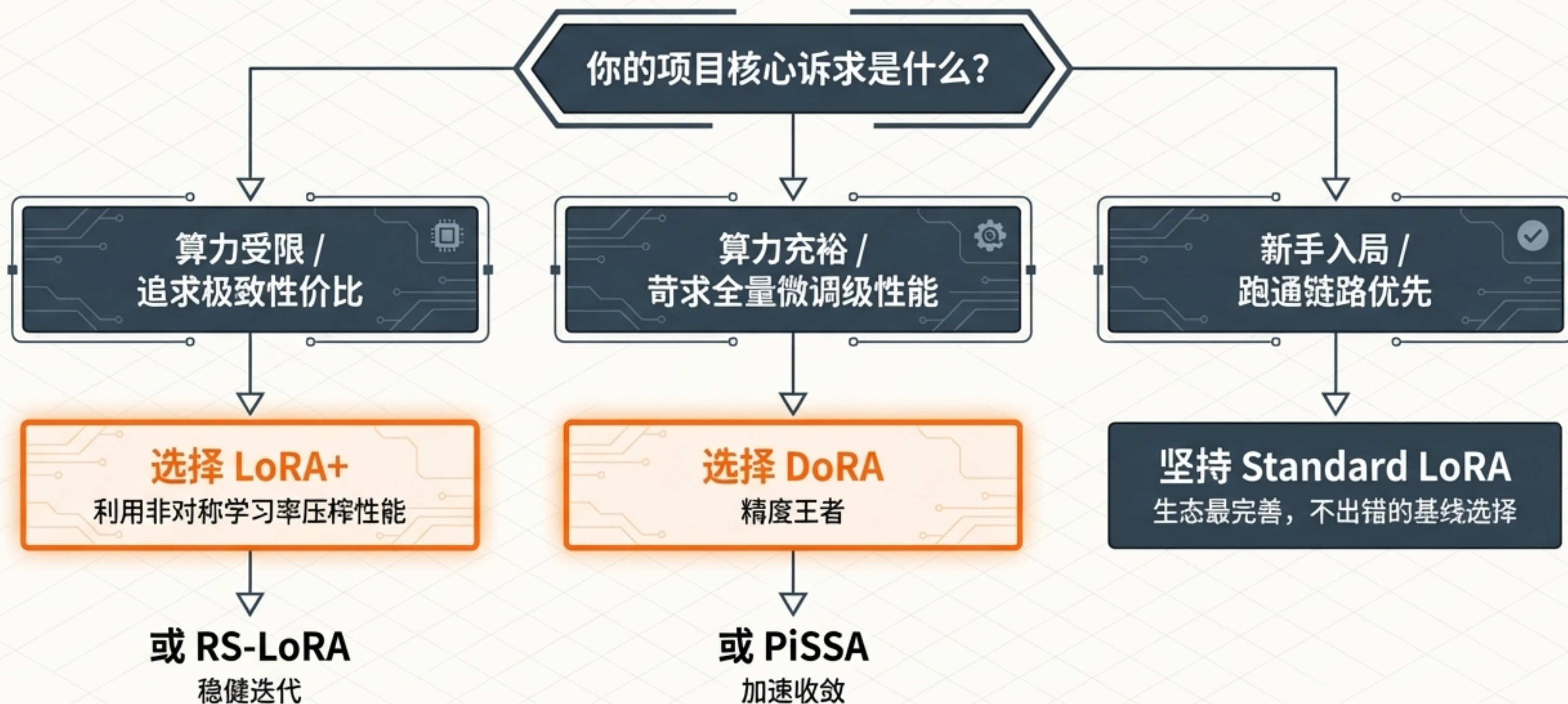


# 前沿进化：四大变体对比矩阵

| 变体名称    | 核心机制                                       | 解决的痛点 / 优势                         |
|---------|--------------------------------------------|------------------------------------|
| LoRA+   | 为 B 矩阵设置更高学习率（通常是 A 的 4-16 倍）。             | 神经网络靠后的输出层对任务变化更敏感。通过非对称学习率压榨极限性能。 |
| DoRA    | 将权重分解为「幅度（Magnitude）」与「方向（Direction）」独立训练。 | 效果最逼近全量微调，突破标准 LoRA 的表达极限，但稍微复杂。   |
| RS-LoRA | 引入 $1/\sqrt{r}$ 缩放因子。                      | 彻底解决高 Rank (r) 值设定下的训练震荡与不稳定性。     |
| PiSSA   | 利用 SVD 主奇异值直接初始化 A 和 B（取代随机/全零）。           | 收敛极快，起步即逼近最优解，但初始阶段计算开销略大。         |



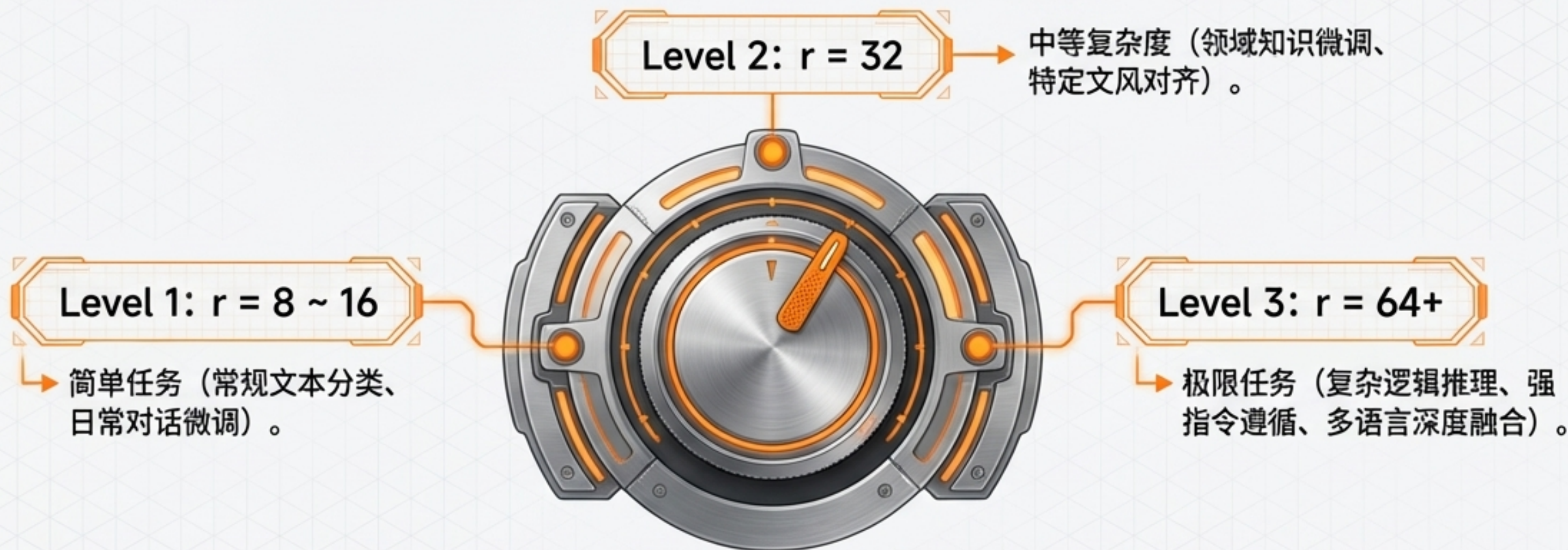
# 实操指南：变体选择决策树





# 核心参数调控台：矩阵秩 (r)

核心定义：秩 (r) 直接决定降维矩阵 A 和 B 的大小，掌控模型的「表达能力容量」与「显存开销」。



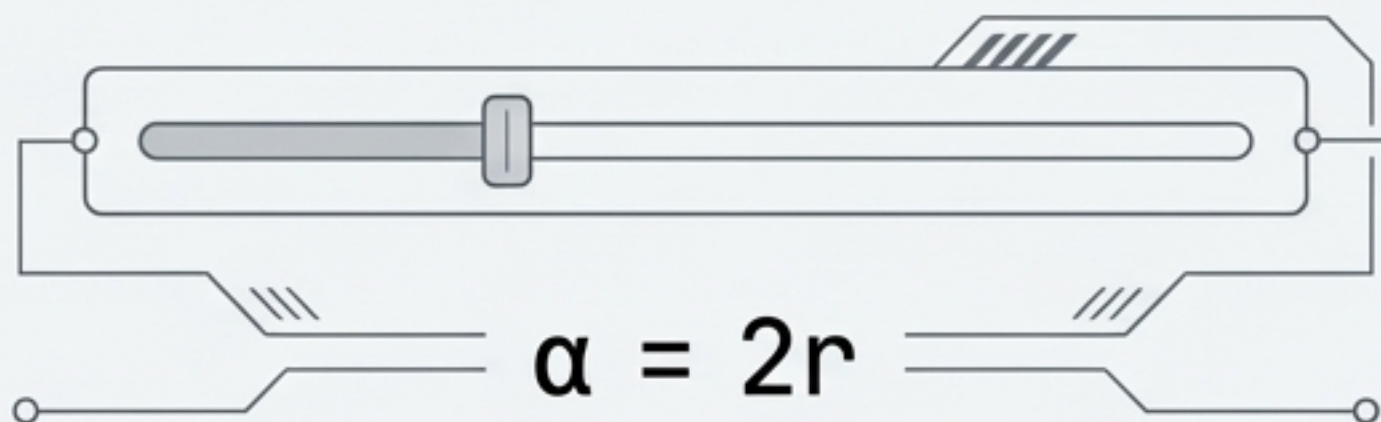
**Warning Label**

工程铁律：在满足性能下限的前提下，尽一切可能使用最小的  $r$ 。  
盲目推高  $r$  只会徒增过拟合风险与计算灾难。



# 进阶参数：缩放系数与目标层

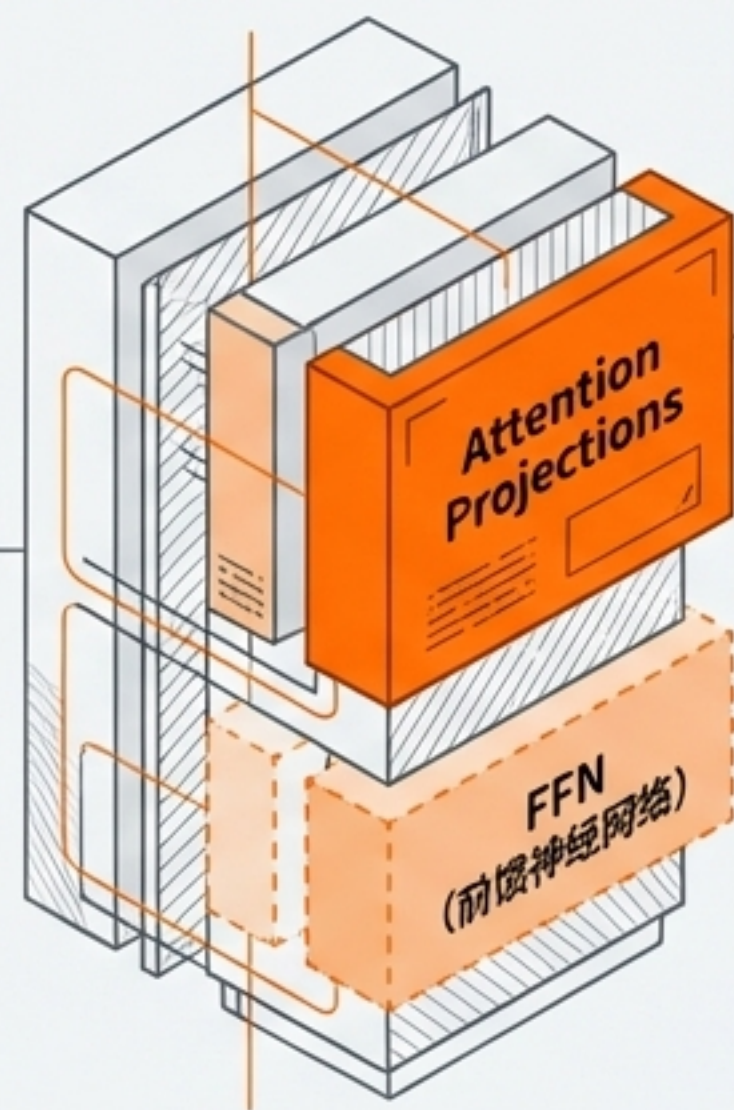
## 控制台 1: Alpha ( $\alpha$ ) 缩放系数



功能：控制增量矩阵  $\Delta W$  对原模型的「干涉强度」。

最佳实践： $\alpha$  经验值通常设为  $r$  的1到2倍（例如  $r=16$ ， $\alpha=32$ ）。手动微调比依赖框架默认行为更稳定。

## 控制台 2: Target Layers (注入目标层)



默认基线：仅应用于**注意力机制的投影层**。对于**80%**的任务已然足够。

深度拓展：当面对**高难度逻辑推理**任务时，将LoRA注入范围扩展至**FFN层**。建议逐步开启并监控显存。



# 高阶实战：极致精度的组合拳 (The Pro-Tuning Formula)

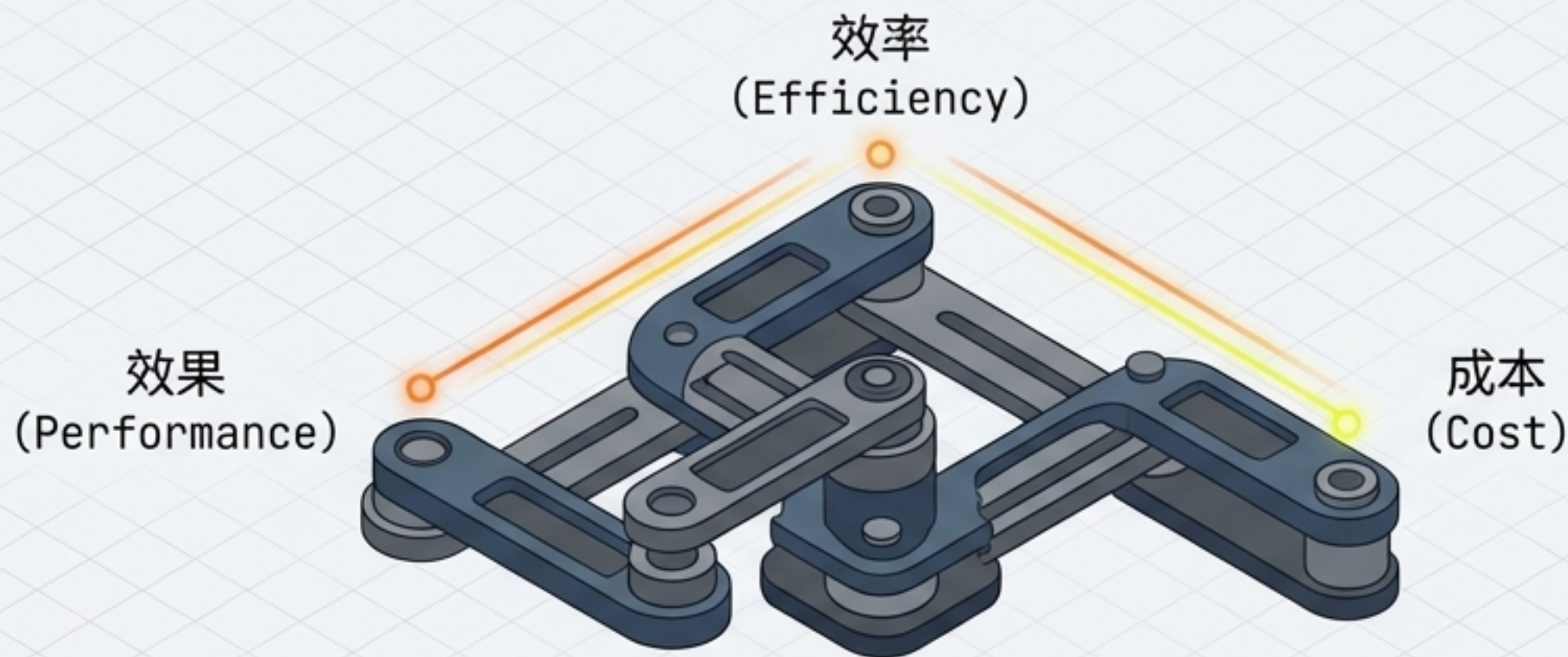
注：其余参数（Learning Rate, Dropout）无需魔改，完全沿用传统全量微调的成熟经验即可。



逼近 99.9% 全量微调精度的终极形态



# 总结：四两拨千斤的艺术



LoRA 在大模型时代找到了开发落地中的极致最优解。

与其推倒重建，不如精准修补。

最终断言：掌握 LoRA，绝不仅仅是多学了一个算法名词，而是掌握了让大模型以最低成本真正在业务中落地的万能钥匙。无论是求职面试还是企业级部署，这都是不可或缺的基石。